
EMBARCADERO CONFERENCE
PATROCINADOR MASTER:
embarcadero
An Idera, Inc. Company

ALESSANDRO MEDEIROS



Analista de Desenvolvimento de Sistemas na TOTVS e Embarcadero MVP, com mais de 20 anos de experiência em Delphi. Atua também com Java, C#/.NET, React e automação fiscal, e é consultor em modernização de sistemas legados com IA.

SOLID | CLEAN ARCH | LOOP ENGINEERING | DEXT.AI | OPENCLAW

Clean Architecture Encontra IA

SOLID, Design Patterns e Loop Engineering aplicados a agentes LLM em Delphi

"Código de IA bagunçado é pior que código legado bagunçado."



Alessandro Medeiros · Embarcadero MVP · Analista de Desenvolvimento · TOTVS

O código que todo mundo já escreveu

```
procedure TFormFinanceiro.btnConsultarClick(Sender: TObject);
var
  HTTP: TNetHTTPClient;
  Response: IHTTPResponse;
  JSON: TJSONObject;
begin
  HTTP := TNetHTTPClient.Create(nil);
  HTTP.CustomHeaders['x-api-key'] := 'sk-ant-api03-XXXX';
  Response := HTTP.Post('https://api.anthropic.com/...');
  JSON := TJSONObject.ParseJSONValue(Response.ContentAsString)
    as TJSONObject;
  MemoResultado.Lines.Add(JSON.GetValue('content').Value);
  // ...e a regra de negócio do financeiro, misturada aqui
end;
```

Funciona. O problema aparece quando o projeto cresce — e com IA cresce rápido.

Quem já escreveu algo assim?

SOLID violado em 13 linhas

S	**Single Responsibility**	✕	1 método faz UI, HTTP, parsing e regra de negócio do financeiro.
O	**Open/Closed**	✕	Trocar de Claude para OpenAI = reescrever o método inteiro.
L	**Liskov Substitution**	✕	Não existe abstração alguma — nada aqui é substituível.
I	**Interface Segregation**	✕	Nenhuma interface segregada: tudo acoplado num único bloco.
D	**Dependency Inversion**	✕	TFormFinanceiro depende direto da classe concreta HTTP.

01

DEPENDENCY INVERSION

Seção 01 — Dependency Inversion

O princípio que separa política de implementação — e abre a porta para testar, trocar e escalar.

Dependência direta vs. invertida

✕ DIRETA

```
TFinanceiroService
        ↓
OpenAI REST API
```

Acoplamento direto ao provedor. Trocar de fornecedor = reescrever toda a classe de serviço.

✓ INVERTIDA

```
TFinanceiroService
        ↓
ILLMProvider
        ↓
implementações concretas
```

"Em testes: TMock. Em dev: TOllama. Em prod: TClaude. O serviço não sabe — e não precisa saber."

Teste unitário com mock

```
[Test]
procedure TestAgenteRespondeCorretamente;
var
  MockProvider: TMockLLMProvider;
  Agent: TReActAgent;
  Response: string;
begin
  MockProvider := TMockLLMProvider.Create(
    'Resposta simulada');
  Agent := TReActAgent.Create(MockProvider,
    FRegistry);
  Response := Agent.Run('Pergunta de teste');
  Assert.AreEqual('Resposta simulada', Response);
end;
```

12ms

Tempo de execução

Sem API key

Nenhum custo envolvido

Determinístico

Mesmo resultado sempre

Isso é o que Dependency Inversion compra para você.

02

SRP + STRATEGY + CLEAN ARCHITECTURE

Seção 02 — SRP + Strategy + Clean Architecture

Separar responsabilidades não é burocracia — é o que permite mudar uma peça sem quebrar as outras.



SEÇÃO 02 · SRP + STRATEGY

Separando responsabilidades

TLLMProvider

comunicação com o provedor de IA

TPromptBuilder

construção do prompt e do system message

TToolRegistry

registro e execução das tools via RTTI

TAgentGraph

orquestração do ReAct loop

TAgentMemory

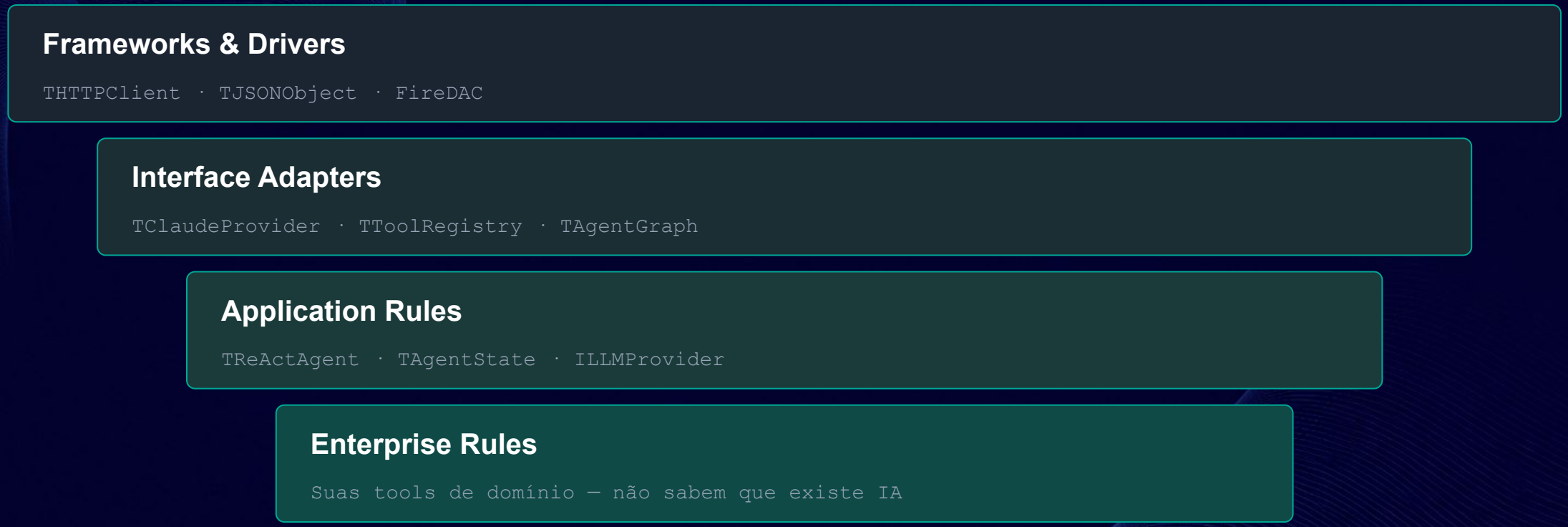
histórico e estado da conversa

TRetryPolicy

política de retry e backoff

TReActAgent — Facade que coordena todas as peças acima.
Mudar retry não toca no provider. Cada peça independente.

Clean Architecture aplicada ao Dext.AI



Regra de ouro: "Setas de dependência apontam para dentro."

"Sua tool pode ser chamada por botão, job noturno, teste ou LLM — ela não sabe qual."

03

LOOP ENGINEERING

Seção 03 — Loop Engineering

De Prompt Engineering para Loop Engineering — e por que Clean Architecture é o harness do loop.

SEÇÃO 03 · LOOP ENGINEERING

A evolução do dev loop — 3 níveis

Nível 1

ReAct Loop

Você dá um prompt. O agente fica em loop até fechar a tarefa.

Ciclos de minutos

Nível 2

Spec Driven

Você define uma spec, que dispara vários loops encadeados: Plan → Implement → Validate.

Ciclos de horas

Nível 3

Loop Engineering

Uma camada acima da spec: o loop decide o próximo spec a executar. O humano define intenção e stop condition.

Ciclos de dias

Loop Fixo vs. Loop Criador

Loop Fixo — sem side effect

Automações, benchmarks, avaliação de frameworks.

Normalmente seguro de rodar de forma autônoma.

Loop Criador — com side effect

Gera roadmap → implementa → gera o próximo.

Ex.: Bun migrou 500 mil linhas para Rust com esse padrão. Requer stop condition confiável.

"Quanto melhor o harness, melhor o loop. A diferença entre caos automatizado e progresso automatizado."

Delphi é um dos melhores harnesses que existem

"Tipagem é harness. Arquitetura é harness. Compilador é harness."

1	**Tipagem forte estática**	O compilador barra o erro antes que ele se perpetue no loop.
2	**Interfaces obrigatórias**	ILLMProvider define o contrato — o loop só chama o que existe.
3	**[AITool] declarativo via RTTI**	Schema JSON gerado automaticamente — o LLM não inventa parâmetros.
4	**TAgentState imutável**	Um bug numa iteração não corrompe as próximas.

SEÇÃO 03 · LOOP ENGINEERING

Checklist: IA em legado, sem bagunçar mais

"Harness é o conceito. Na prática, vira regra."

1	**Contrato antes de código**	Decisão sem contrato explícito é ambiguidade escondida — e ambiguidade escondida é a IA decidindo por você.
2	**Harness primeiro, geração depois**	Sem teste ou compilador travando erro, não é hora de gerar código ali.
3	**Extrair antes de reescrever**	Uma responsabilidade por vez, valida, avança.
4	**Escopo é do humano**	A IA sempre vai querer fazer mais do que foi pedido.
5	**Contrato público muda → review humano**	Sem exceção.

O mesmo problema, do jeito certo

✕ ANTES

80 linhas dentro do OnClick, misturando UI, HTTP e regra de negócio.

✓ DEPOIS

```
var R := FAgent.Run(MemoInput.Text);
```

3 linhas.

Testável · Trocável · Loop-safe · Extensível

Vamos ver isso rodando ao vivo: um ERP legado ganhando um agente orientado a grafo — diagnosticando um problema fiscal e pedindo aprovação antes de agir.

BLOCO 3 · DEMO

Vamos ao Demo



04

SOFTWARE FACTORY MULTI-AGENTE

Isso é só o começo.

O que você viu aqui é a fundação. Loop Engineering em escala industrial já está rodando.

SEÇÃO 04 · SOFTWARE FACTORY MULTI-AGENTE

Agentes especializados, trabalhando entre si

Um padrão que dá para construir com qualquer stack de IA — esta é a implementação que eu fiz, com OpenClaw.

M	V	L	F	C
MAESTRO	**VETOR**	**LAYOUT**	**FISCO**	**CRIVO**
Orquestrador · Delega · Decide	Backend · Delphi/Java · Arquitetura	Frontend · React/Vue · UI/UX	ERP/Fiscal · NF-e · eSocial	QA · testes · ativa após CI green

Cada agente tem: soul file · identity file · tools file — a mesma ideia funciona com qualquer orquestrador.

Regra invariante no meu approach: "manual merge always" — nunca merge automático.

COLEÇÃO NES · VOLUME 1

Tudo isso está formalizado no livro

Os princípios que citei a palestra inteira — não são slides soltos, são capítulos.



Dos princípios fundamentais da NES ao Software Factory Cognitivo — o mesmo raciocínio que sustentou cada slide de hoje, explicado a fundo, capítulo por capítulo.

alemedeiros.com.br/livros

Escaneie e acesse a página do livro

Perguntas?

Alessandro Medeiros

Analista de Desenvolvimento de Sistemas · TOTVS · Embarcadero MVP

alemedeiros.com.br/livros · github.com/cesarliws/dext · @alepmedeiros

SOLID | CLEAN ARCH | LOOP ENGINEERING | DEXT.AI | OPENCLAW

Obrigado!

EMBARCADERO CONFERENCE 26

PATROCINADOR MASTER: embarcadero® An Idera, Inc. Company

QUER ME VER NA #ECON27?

ACESSE O QRCODE E AVALIE MINHA PALESTRA!

NOME DO PALESTRANTE

@alemedeirosp

linkedin.com/in/alepmedeiros

alemedeiros@bsd.com.br

alemedeiros.com.br

(21) 9 9300-5210